

that you access the resulting objects is exactly the same. The “Don’t Repeat Yourself” (DRY) principle says that having these duplicated methods is a bad idea, because any bug fixes and/or enhancements would have to be applied in both copies of the methods.

Instead, this is a good opportunity to merge classes. We can create a common abstract base class for these classes to inherit from. The base class will have its own `__init__()` method that includes any common code from the `__init__()` methods of both original classes, and it will contain the `play()`, `update()`, and `draw()` methods.

Each original class will inherit from the new base class and implement its own `__init__()` method using the appropriate parameters. Each will do its own work to create `self.imagesList`, which is then used in the other three methods in the new base class.

Rather than showing the result of merging these two “simple” classes, I’ll show the result of this merge in the “professional strength” `Animation` and `SpriteSheetAnimation` classes that are part of the `pygwidgets` package.

Animation Classes in pygwidgets

The `pygwidgets` module contains the following three animation classes:

`PygAnimation` An abstract base class for the `Animation` and `SpriteSheetAnimation` classes

`Animation` A class for image-based animations (separate image files)

`SpriteSheetAnimation` A class for sprite sheet–based animations (a single large image)

We’ll look at each class in turn. The `Animation` and `SpriteSheetAnimation` classes use the same basic concepts discussed, but also have more options available via initialization parameters.

Animation Class

You use `pygwidget`’s `Animation` class to create an animation from many different image files. Here is the interface: